

Beginner's Guide: Go + Redis with the Keploy Quickstart

Turn real API calls into automatic tests. Gin + Redis sample app, Keploy 3.8.58, Windows with WSL2. Real screenshots, real bumps, and a few interesting facts along the way.

What you will do. Run a small Go app, let Keploy watch it while you use it, and get automatic tests out of that. Then switch Redis off and watch the tests still pass. **You will write zero test code.**

Time: about an hour if everything goes smoothly, longer if you collect bugs like I did. **You need:** Windows 10 or 11 and the ability to copy and paste commands. That is it.

Status of this document: record and replay are done (5 of 5 tests pass).

In this guide

1. Why Keploy?	9. Record: let Keploy watch
2. Jargon decoder	10. What Keploy wrote
3. My setup	11. Replay: the stunt double steps in
4. Meet the sample app	12. Your turn: break it on purpose
5. Installing Keploy	13. Cheat sheet
6. Getting Go and Docker working	14. Troubleshooting log
7. Clone the app and read the code	15. Why Keploy earned a thumbs up
8. Start Redis, build the app	Appendices: figures, website plan

1. Why Keploy? The two-minute version

Normally, testing an API means writing test code by hand: build a request, call your handler, check the answer. And if your service talks to Redis or a database, you also have to fake that part, either with a throwaway Redis or with hand-written mocks you then have to keep up to date. It is valuable work. It is also a lot of typing.

Keploy flips the order. **You use your app the normal way, and Keploy takes notes.** Think of a dashcam: it records the drive, and later you can replay the footage and check that nothing changed. Keploy does that for your API calls, *and* for the Redis traffic behind them.

Interesting fact. Redis stands for **RE**remote **D**ictionary **S**erver. It keeps its data in memory, which is why it is so fast and why it is a classic home for short-lived data like one-time passwords. It is also exactly the kind of dependency that makes hand-written tests tedious.

Here is how it works, in one picture:

```
RECORD MODE (you use the app, Keploy watches)

  curl / Postman ---> [ your Go app ] ---> Redis
                        ^               ^
                        |               |
                        |___ Keploy sits in the middle and saves:
                            1) each request and the answer your app gave -> tests/*.yaml
                            2) each message to Redis and Redis's reply   -> mocks.yaml

TEST MODE (Keploy plays the footage back)

Keploy ---> [ your Go app ] ---> Keploy's "stunt double" for Redis (the saved mocks)
|                                     no real Redis needed
|--- sends the saved requests, then compares the app's answers with the saved ones
```

Without Keploy vs with Keploy, for a Go service that uses Redis:

	The hand-written way	The Keploy way
Writing tests	Build requests and assertions in Go, case by case	Make the calls with curl; Keploy writes the test files
Redis in tests	Run a real Redis, or fake it in code	Keploy saves Redis's replies and plays them back (I proved it with Redis switched off)
Changing your code	Often means restructuring so dependencies can be swapped for fakes	I changed none of the app's code to record it
Test data	Examples you invent	Real requests and real responses from your running app
Reading the tests	Go code	Plain YAML you can open in any editor

Why Go developers in particular will like it

- **It works with the binary you already build.** `go build`, then `keploy record -c "./your-app"`. No SDK to import.
- **It sees past HTTP.** It captured the app's Redis conversation too, not just the requests: I got 6 Redis mocks including the exact write that stores the OTP.
- **The tests are readable.** Plain text files you can read, diff and review. No magic binary blobs.
- **The logs talk to you.** You get a clear "Keploy has captured test cases" line each time, and when something stalled, the debug log said exactly where. That is how I found the problem in section 9.
- **The free plan was enough.** Everything in this guide ran on the Keploy Free plan.
- **It is quick.** My replay of 5 tests finished in about 20 seconds, most of that being the startup delay I told it to wait.

Heads up. I promised honest notes, so here is the fair warning. On Windows, Redis support meant using WSL. Random values need "noise" handling. And the recorder gave me a hard time once (section 9). None of that was a deal-breaker, and each bump taught me something about how Keploy works, so I kept them in.

2. Jargon decoder

Skim this now, come back when a word trips you up.

Word	Plain English
API	The doors of your app. You knock (send a request) and get an answer back.
Test case	One saved request plus the answer your app gave. Keploy stores it as a small text file.
Mock	A saved copy of a dependency's answer (here, Redis). Think of it as a stunt double for Redis.
Record / test	Record: you use the app and Keploy takes notes. Test: Keploy replays the notes and checks the answers match.
Noise	Fields that change on every run (random numbers, dates, tokens). Keploy is told to ignore them when comparing.
Proxy	A middleman that passes messages along. Keploy acts as one so it can take notes in between.
Redis	A very fast in-memory store. In this app it remembers each one-time password for a while.
OTP	One-time password: a short code, here 4 digits.
JWT	A signed token the app hands back after login. It contains an expiry time, which matters later.
WSL2	A real Linux running inside Windows. A Linux apartment inside your Windows house.
Container	A small sealed box that runs a program. We run Redis in one so we do not have to install it.

3. My setup

Item	Value
OS	Windows, with WSL2 Ubuntu 22.04.5 LTS (kernel 6.18.33.2-microsoft-standard-WSL2; Keploy needs 5.10+)
Go	1.27.1 (installed with snap)
Docker	Docker 29.7.2, Compose v5.4.0 (Docker Desktop, WSL integration on)
Git	2.34.1
Keploy	3.8.58, Free plan
Redis	Unpinned image first (Redis 8.x), later redis:7 (7.4.11)
Sample app	keploy/samples-go, folder gin-redis

4. Meet the sample app: a tiny bouncer

Keploy's Quickstarts page is a three-step picker: language, environment, then quickstart. I chose Go and landed on **Gin + Redis**. It is a small authentication app that works like a bouncer with a very short memory:

- You give it your email: it makes up a 4-digit code (the OTP), stores it in Redis, and tells you the code.
- You come back with that code: it looks up what Redis remembers, compares, and if it matches, hands you a token.

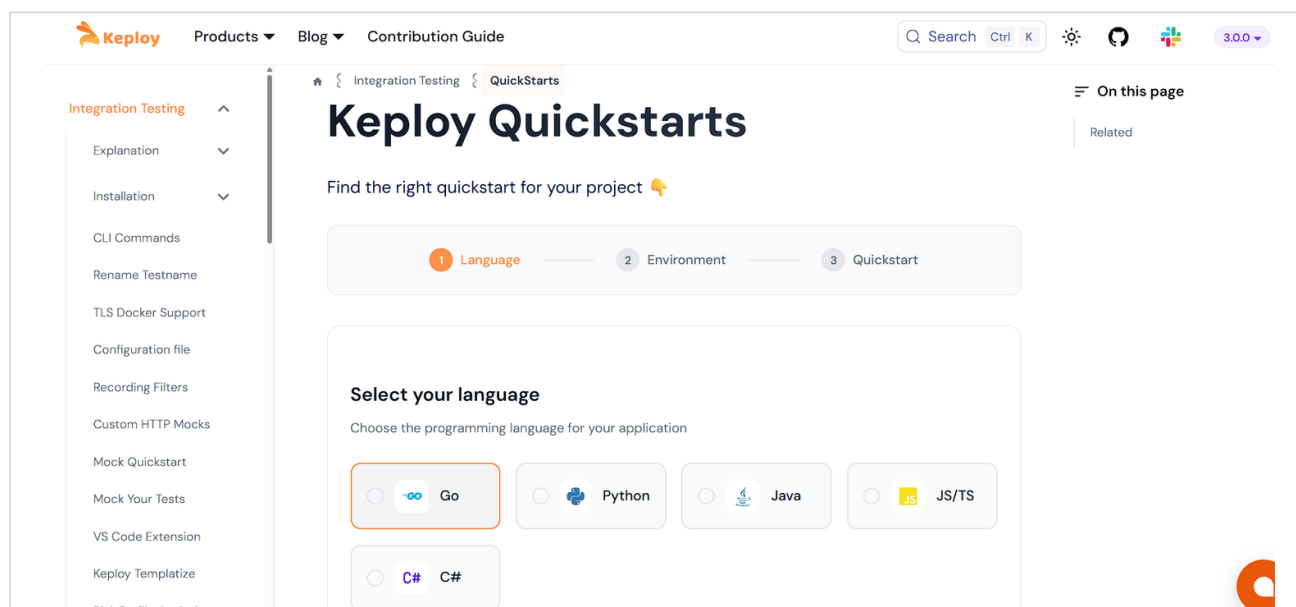


Figure 1. Quickstarts page, step 1: select the language (Go).

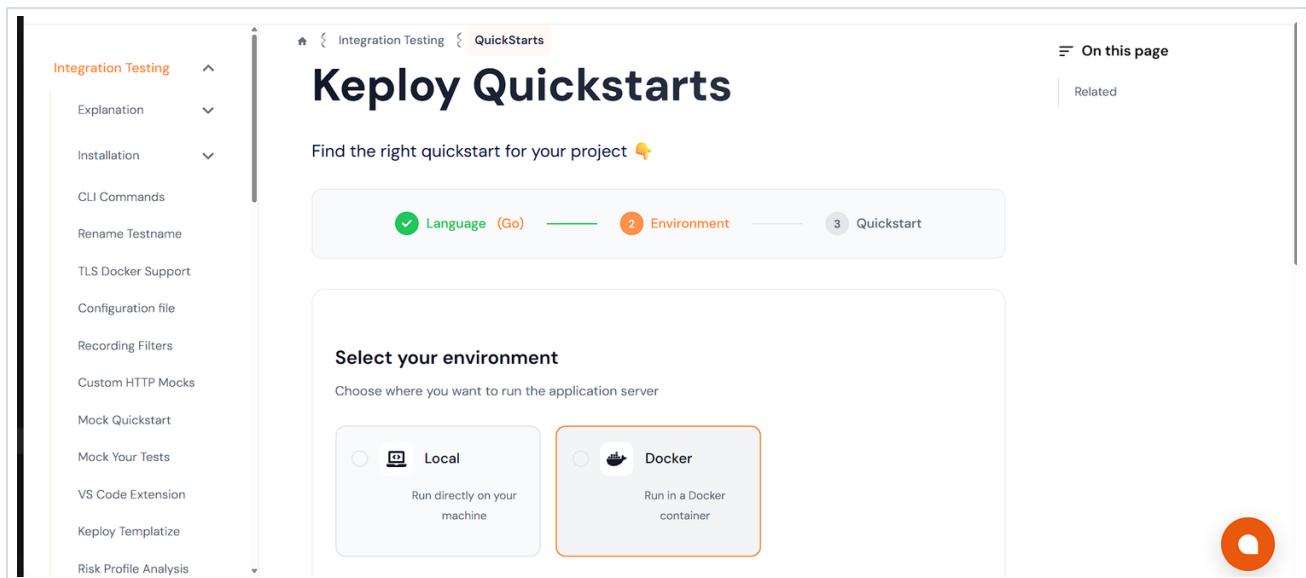


Figure 2. Step 2: pick where the app runs (Local or Docker). My own run used the app locally in WSL with Redis in Docker.

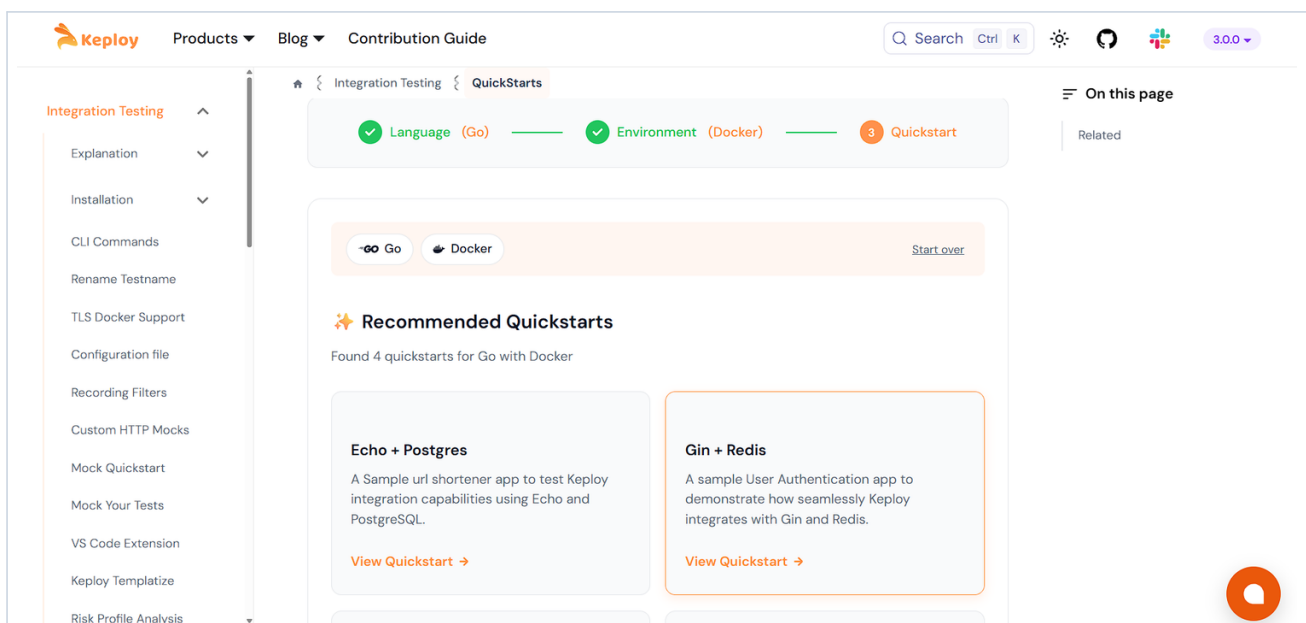


Figure 3. The Go list: "Found 4 quickstarts for Go with Docker", including Gin + Redis.

Why this matters. Redis is a dependency, a thing your app talks to. Dependencies are exactly what make tests painful, and exactly what Keplay is good at. Picking an app with one makes the demo meaningful.

Why WSL. Keplay's install docs say that natively on Windows it understands HTTP/HTTPS, MySQL and MongoDB, while services like Redis are captured only as raw bytes and usually do not replay. So I ran everything in WSL2 (Linux). I also did not follow the quickstart's Docker run path exactly: I ran the app directly in WSL with Redis in Docker.

5. Installing Keplay

5.1 First try: native Windows (and why I bailed)

Keplay's Windows page offers three methods: native, WSL, or Docker. I started with the native PowerShell script, which downloads `keplay.exe` into `%USERPROFILE%\keplay\bin`.

1. Install Keploy

Run this in **PowerShell** — a normal one; Keploy does not need Administrator. It downloads `keploy.exe` into `%USERPROFILE%\keploy\bin` and adds that folder to your user `PATH`:

```
$ProgressPreference = 'SilentlyContinue'
[Net.ServicePointManager]::SecurityProtocol = [Net.ServicePointManager]::SecurityProtocol -bor [Net.SecurityProtocolType]::Tls12
$dir = "$env:USERPROFILE\keploy\bin"
New-Item -ItemType Directory -Force $dir | Out-Null
Invoke-WebRequest -Uri "https://keploy.io/ent/dl/latest/enterprise_windows_amd64.exe" -
-OutFile "$dir\keploy.exe"
Unlock-File "$dir\keploy.exe"
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")
if ($userPath -notlike "**$dir*") {
    [Environment]::SetEnvironmentVariable("Path", "$userPath;$dir", "User")
}
```

The file is named `enterprise_windows_amd64.exe` for historical reasons — it is the `keploy` CLI, and a free account is all you need.

Figure 4. Keploy docs: the native Windows install script.

Figure 5. The download finished (213,541,888 bytes), but seconds later Unblock-File says the file does not exist. Windows Security had flagged it with a Trojan warning and removed it.

My decision. I did not add an exclusion or allow the file. I could not verify whether the detection was a false positive, so I took the other road: WSL2.

5.2 Second try: WSL2 (this one worked)

Open PowerShell as Administrator and run:

```
wsl --install -d Ubuntu-22.04
```

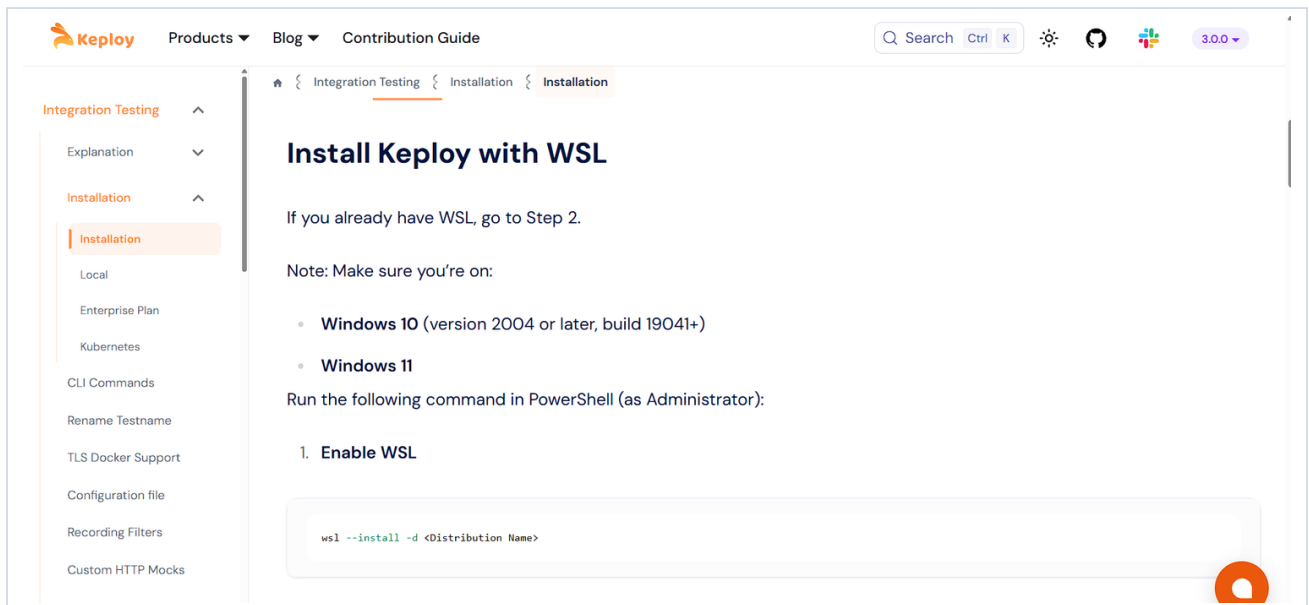


Figure 6. Keptel docs: Install Keptel with WSL.

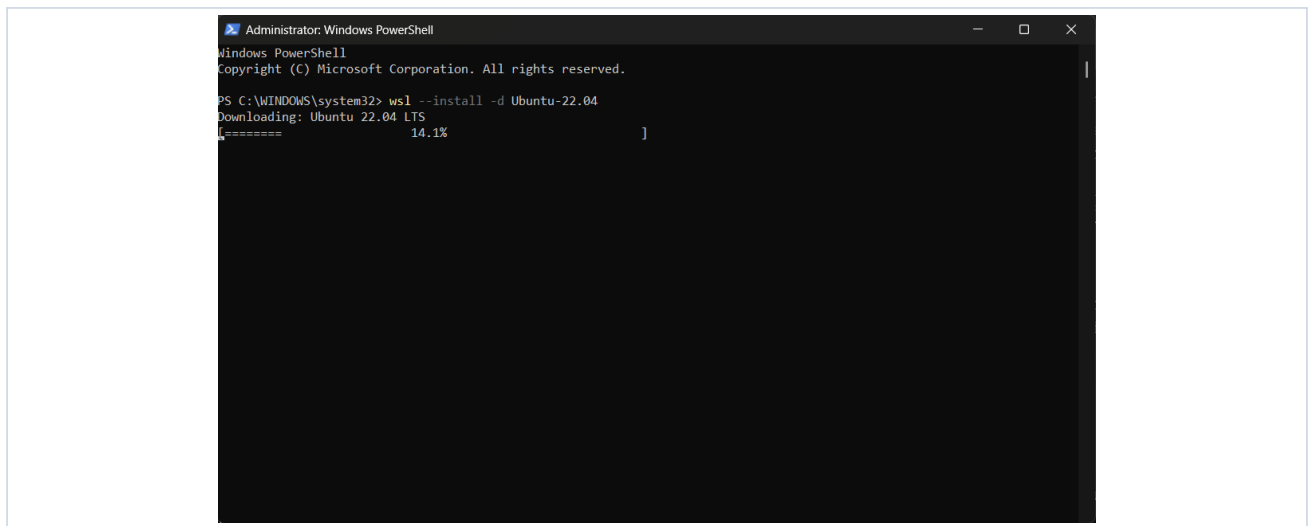


Figure 7. Downloading the Ubuntu-22.04 distribution (14.1% and climbing).

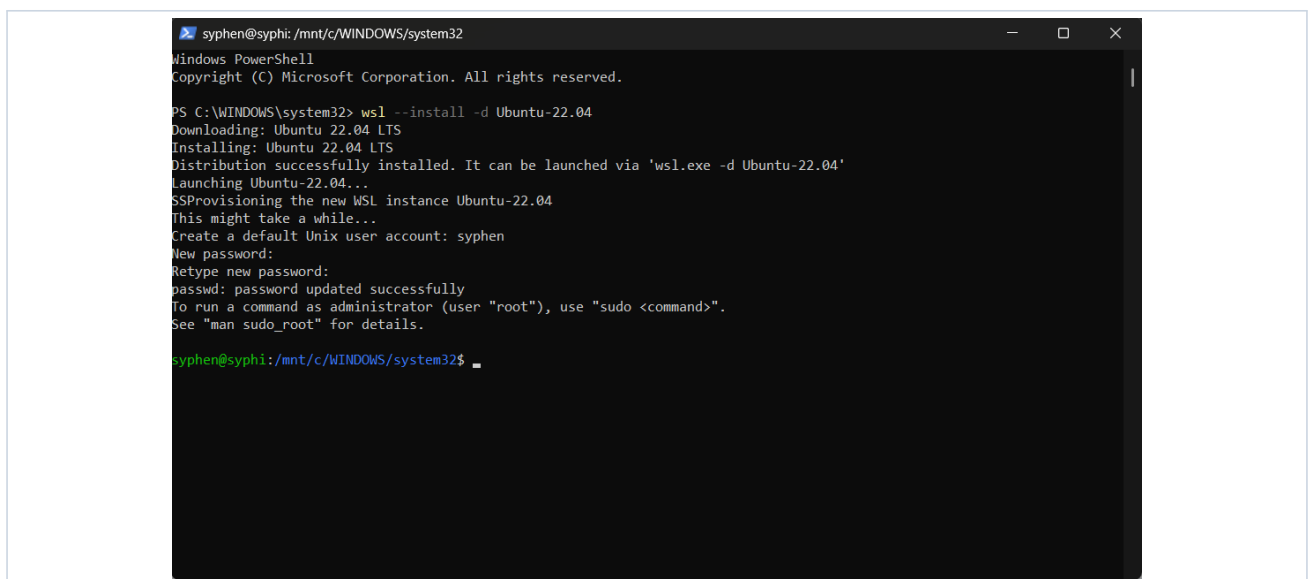


Figure 8. Ubuntu installed and a default user created.

Newbie tip. WSL is a full Linux inside Windows, so you now have two worlds: Windows (PowerShell) and Linux (the Ubuntu terminal). Every command from here on runs in the **Ubuntu** terminal, and your project lives in the Linux home folder (~), not in a Windows folder like /mnt/d/. . . . Keploy refuses to write its log file on Windows drives with a "chmod ... operation not permitted" error.

Interesting fact. WSL2 runs a real Linux kernel (mine is 6.18.33.2). Keploy needs Linux kernel 5.10 or newer, so WSL2's real kernel is a good fit.

Now install Keploy inside Ubuntu:

```
curl --silent -O -L https://keploy.io/install.sh && source install.sh
keploy --version
```

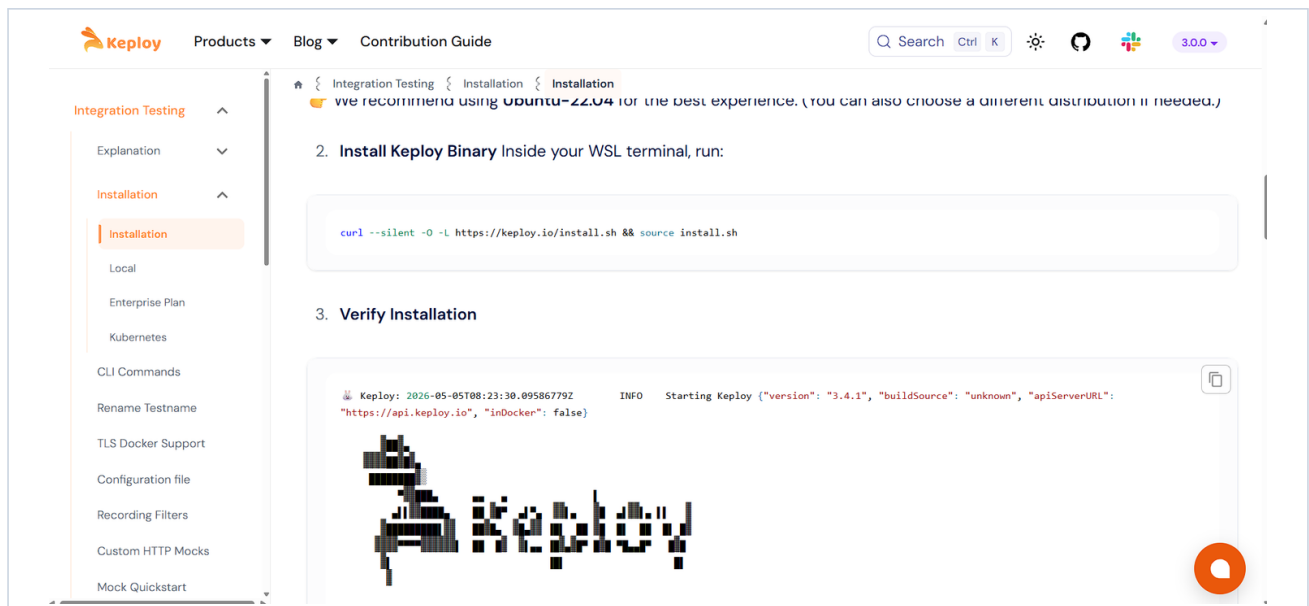


Figure 9. Docs: install the Keploy binary inside the WSL terminal.



Figure 10. Install output: Keploy 3.8.58 and its command list. The "Docker not found on device" line appears because Docker Desktop's WSL integration was not on yet (fixed in section 6).

5.3 Logging in with an API key

Keplay supports browser login, manual login and API-key login. I used an API key: create a token in the Keplay web app (Settings, then API Keys), then log in from the terminal.

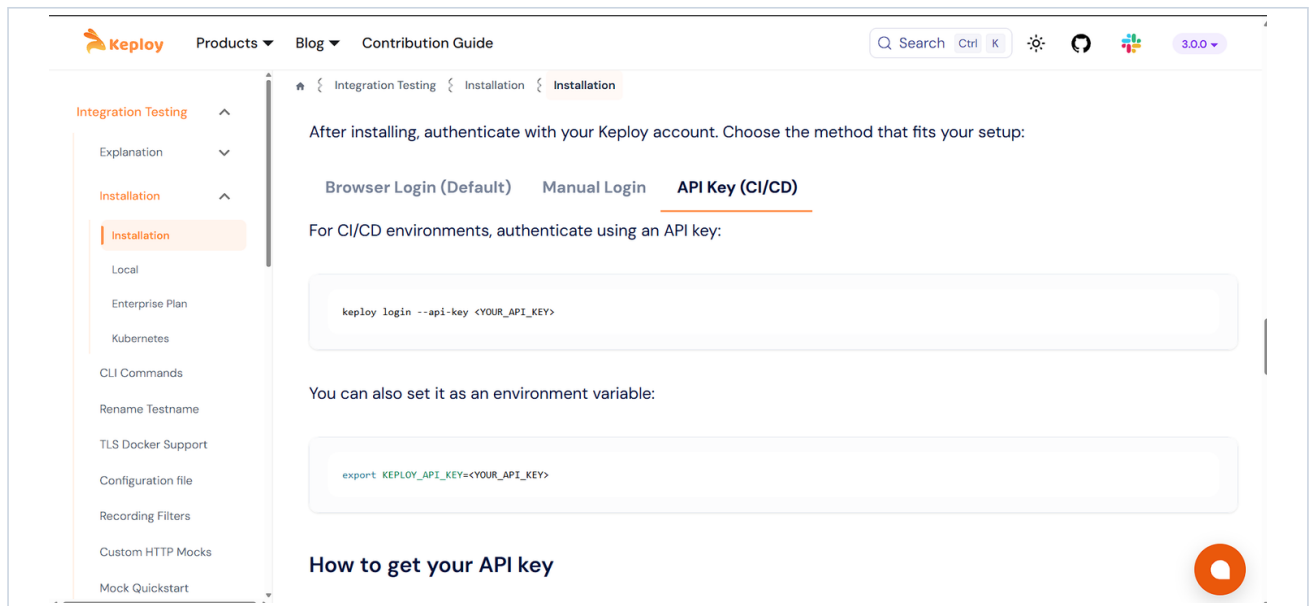


Figure 11. Docs: the three login tabs; I used API Key.

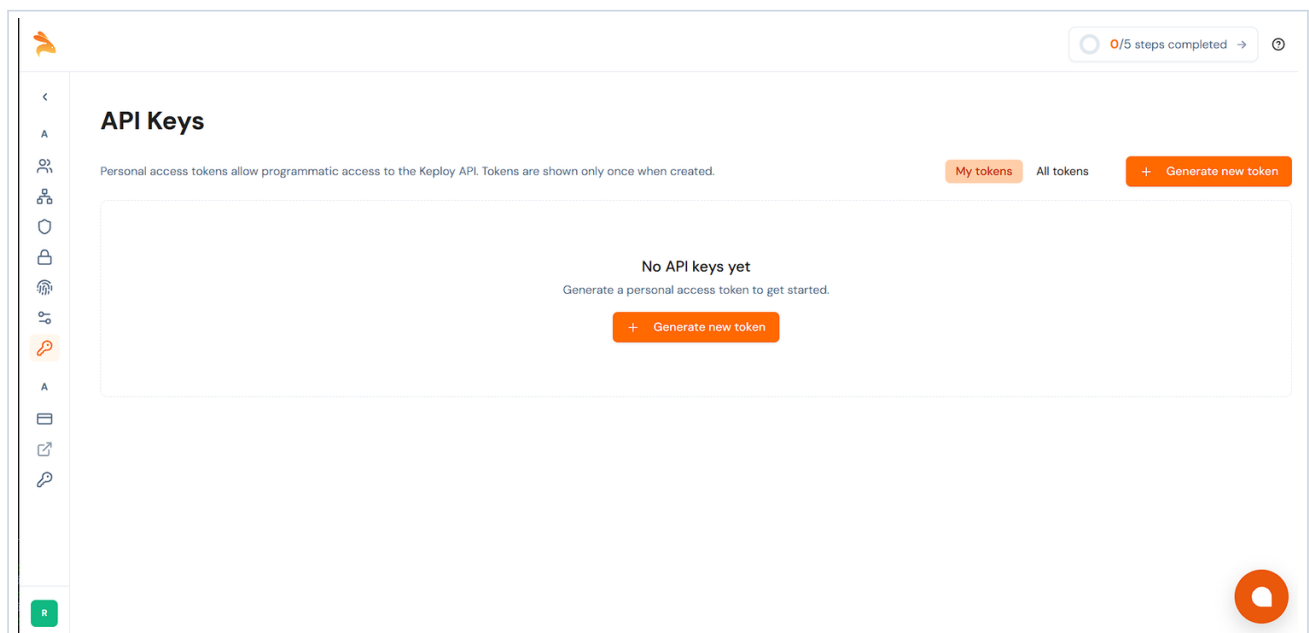


Figure 12. app.keplay.io, Settings, API Keys. Tokens are shown only once, so copy yours when you create it.

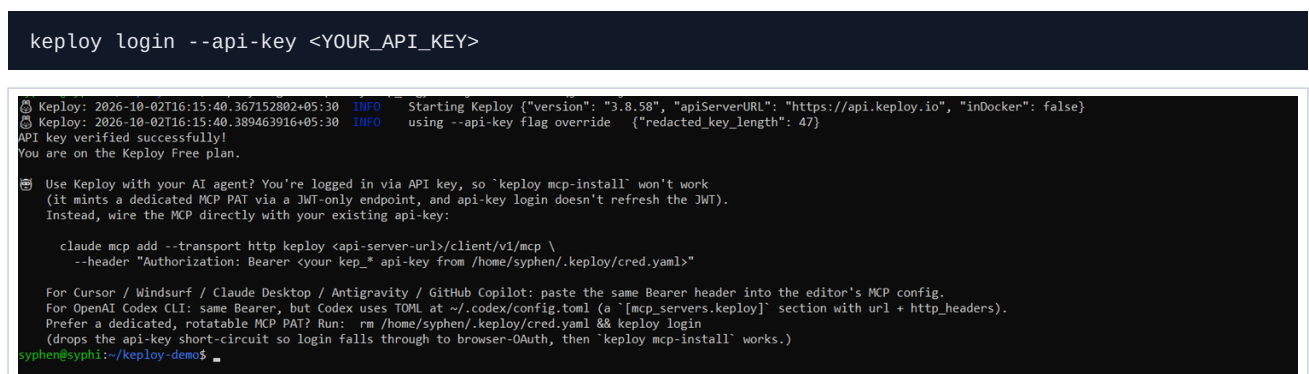


Figure 13. "API key verified successfully" on the Free plan. The log shows the key length, not the key.

6. Getting Go and Docker working in WSL

My first tools check came back with two red flags.

```
(drops the api-key short-circuit so login falls through to browser-0Auth, then keploy mcp
syphen@syphi:~$ go version
docker --version
docker compose version
git --version
uname -r
Command 'go' not found, but can be installed with:
sudo snap install go          # version 1.27.1, or
sudo apt install golang-go    # version 2:1.18~0ubuntu2
See 'snap info go' for additional versions.

The command 'docker' could not be found in this WSL 2 distro.
We recommend to activate the WSL integration in Docker Desktop settings.

For details about using Docker Desktop with WSL 2, visit:
https://docs.docker.com/go/wsl2/

The command 'docker' could not be found in this WSL 2 distro.
We recommend to activate the WSL integration in Docker Desktop settings.

For details about using Docker Desktop with WSL 2, visit:
https://docs.docker.com/go/wsl2/

git version 2.34.1
6.18.33.2-microsoft-standard-WSL2
```

Figure 14. First check: go not found and docker unavailable in this WSL distro.

Symptom	Cause	Fix
go: command not found ("available in /snap/bin/go")	Go came from snap, but /snap/bin was not on PATH	echo 'export PATH=\$PATH:/snap/bin' >> ~/.bashrc, then source ~/.bashrc
docker could not be found in this WSL 2 distro	Docker Desktop's WSL integration is off	Docker Desktop, Settings, Resources, WSL integration: enable Ubuntu; open a new terminal
permission denied on /var/run/docker.sock	Your user is not in the docker group	sudo usermod -aG docker \$USER, then newgrp docker or reopen the terminal

```
syphen@syphi:~$ go version
go version go1.27.1 linux/amd64
syphen@syphi:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
syphen@syphi:~$
```

Figure 15. After the fixes: go1.27.1 and a working (empty) docker ps.

7. Clone the app and read the code first

```
mkdir -p ~/keploy-demo && cd ~/keploy-demo
git clone https://github.com/keploy/samples-go.git
cd samples-go/gin-redis
go mod download
```

```

syphen@syphi:~$ mkdir -p ~/keploy-demo && cd ~/keploy-demo
git clone https://github.com/keploy/samples-go.git
cd samples-go/gin-redis
go mod download
ls
Cloning into 'samples-go'...
remote: Enumerating objects: 6914, done.
remote: Counting objects: 100% (805/805), done.
remote: Compressing objects: 100% (404/404), done.
remote: Total 6914 (delta 597), reused 440 (delta 398), pack-reused 6109 (from 2)
Receiving objects: 100% (6914/6914), 148.02 MiB | 1.57 MiB/s, done.
Resolving deltas: 100% (4345/4345), done.
Dockerfile  README.md  config  constants  controllers  docker-compose.yml  go.mod  go.sum  helpers  img  main.go  middlewares  routes  server  services  structure  utils

```

Figure 16. Clone, go mod download and the project files.

```

syphen@syphi:~/keploy-demo/samples-go/gin-redis$ cat docker-compose.yml~cat main.go
cat: docker-compose.yml~cat: No such file or directory
// Package main initates the server
package main

import (
    "github.com/keploy/gin-redis/server"
)

func main() {
    server.Init()
}
syphen@syphi:~/keploy-demo/samples-go/gin-redis$ cat docker-compose.yml
version: '3.7'
services:
  go-app:
    build:
      context: .
    container_name: ginRedisApp
    ports:
      - "3001:3001"
    depends_on:
      - redis
  redis:
    image: redis
    container_name: myredis
    ports:
      - "6379:6379"syphen@syphi:~/keploy-demo/samples-go/gin-redis$

```

Figure 17. docker-compose.yml: the app on port 3001 and Redis as container myredis on 6379. (The first command in the screenshot has a typo that printed an error; harmless.)

Reading the code before running anything is the cheapest debugging there is. Here is what I learned from it:

- **Port:** 3001.
- **Redis address:** localhost:6379 in helpers/redis/redisConnect.go, so no code change is needed to run the app directly.
- **GET /api/getVerificationCode?email=...&username=...** makes a random 4-digit OTP, stores it in Redis, and returns it.
- **POST /api/verifyCode** takes email (string) and otp (an integer), reads the stored OTP back from Redis, and returns a JWT on a match.
- **Errors:** a missing field gives 422; a wrong OTP gives 500 ("OTP didn't match, email isn't verified").
- **The Redis write Keploy will capture:** SET with the email as the key, the OTP and username as JSON, and EX 14400, a 4-hour expiry.

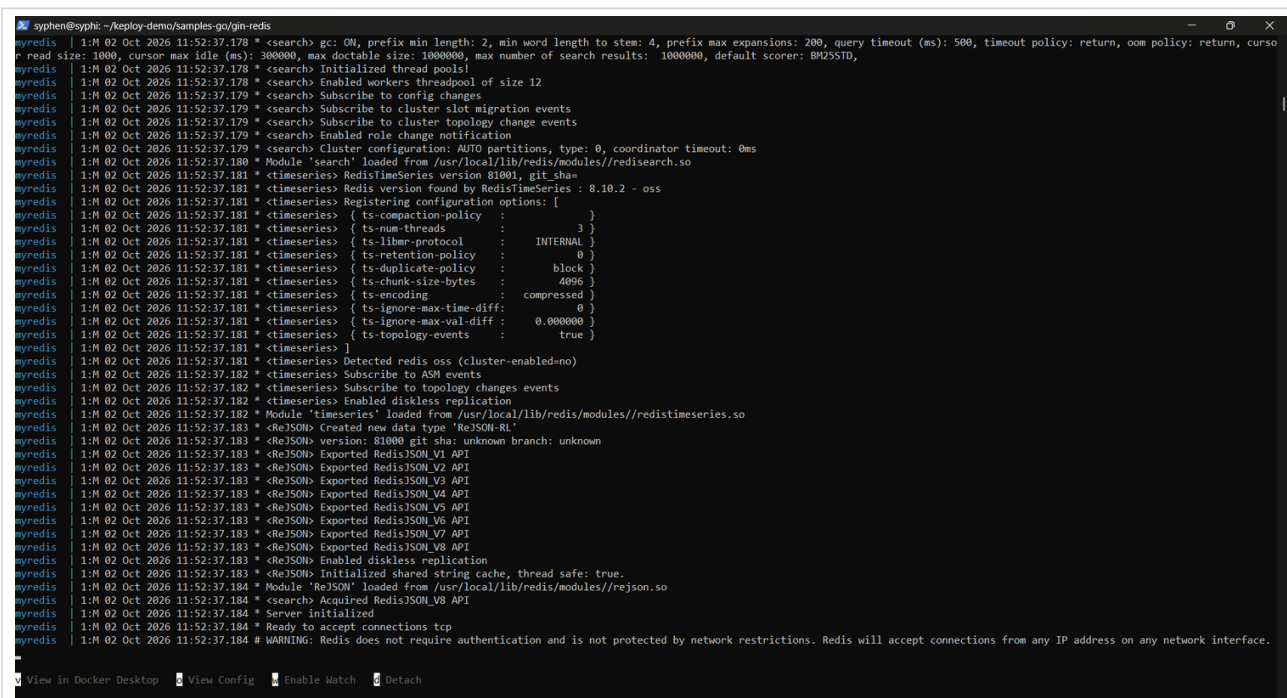
Interesting fact. EX 14400 means the key expires after 14,400 seconds, which is exactly 4 hours. Redis deletes expired keys on its own, so the app never has to clean up old OTPs.

8. Start Redis, build the app

You will use three terminal windows from here on. Think of them as three actors: **Terminal 1** runs Redis, **Terminal 2** runs Keploy and the app, **Terminal 3** is you, sending requests.

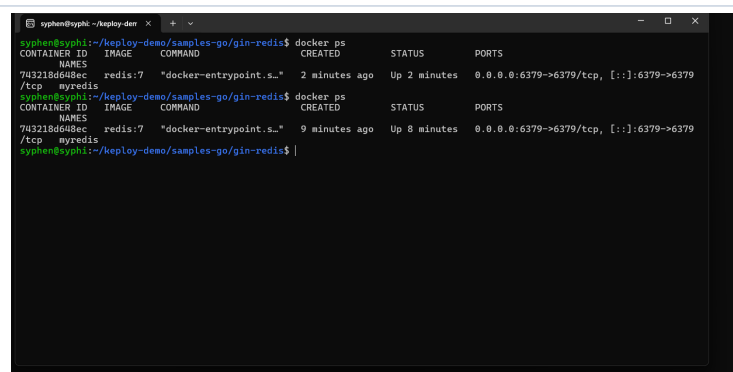
```
# Terminal 1 (leave it running)
docker compose up redis

# Terminal 2
docker ps
go build -o gin-redis
```



```
myredis | 1:M 02 Oct 2026 11:52:37.178 * <search> gc: ON, prefix min length: 2, min word length to stem: 4, prefix max expansions: 200, query timeout (ms): 500, timeout policy: return, oom policy: return, cursor read size: 1000, cursor max idle (ms): 300000, max doctable size: 1000000, max number of search results: 1000000, default scorer: BM25STD,
myredis | 1:M 02 Oct 2026 11:52:37.178 * <search> Initialized thread pools!
myredis | 1:M 02 Oct 2026 11:52:37.178 * <search> Enabled workers threadpool of size 12
myredis | 1:M 02 Oct 2026 11:52:37.179 * <search> Subscribe to config changes
myredis | 1:M 02 Oct 2026 11:52:37.179 * <search> Subscribe to cluster slot migration events
myredis | 1:M 02 Oct 2026 11:52:37.179 * <search> Subscribe to cluster topology change events
myredis | 1:M 02 Oct 2026 11:52:37.179 * <search> Enabled role change notification
myredis | 1:M 02 Oct 2026 11:52:37.179 * <search> Cluster configuration: AUTO partitions, type: 0, coordinator timeout: 0ms
myredis | 1:M 02 Oct 2026 11:52:37.180 * Module 'search' loaded from /usr/local/lib/redis/modules//redisearch.so
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> RedisTimeSeries version 81001, git sha=
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> Redis version found by RedisTimeSeries : 8.10.2 - oss
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> Registering configuration options: [
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-compaction-policy : }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-num-threads : 3 }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-libm-protocol : INTERNAL }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-retention-policy : 0 }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-duplicate-policy : block }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-chunk-size-bytes : 4096 }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-encoding : compressed }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-ignore-max-time-diff : 0 }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-ignore-max-val-diff : 0.000000 }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> { ts-topology-events : true }
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> ]
myredis | 1:M 02 Oct 2026 11:52:37.181 * <timeseries> Detected redis oss (cluster-enabled=no)
myredis | 1:M 02 Oct 2026 11:52:37.182 * <timeseries> Subscribe to ASM events
myredis | 1:M 02 Oct 2026 11:52:37.182 * <timeseries> Subscribe to topology changes events
myredis | 1:M 02 Oct 2026 11:52:37.182 * <timeseries> Enabled diskless replication
myredis | 1:M 02 Oct 2026 11:52:37.182 * Module 'timeseries' loaded from /usr/local/lib/redis/modules//redistimeseries.so
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Created new data type 'reJSON-RL'
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> version: 81000 git sha: unknown branch: unknown
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V1 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V2 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V3 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V4 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V5 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V6 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V7 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Exported RedisJSON_V8 API
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Enabled diskless replication
myredis | 1:M 02 Oct 2026 11:52:37.183 * <reJSON> Initialized shared string cache, thread safe: true.
myredis | 1:M 02 Oct 2026 11:52:37.184 * Module 'reJSON' loaded from /usr/local/lib/redis/modules//rejson.so
myredis | 1:M 02 Oct 2026 11:52:37.184 * <search> Acquired RedisJSON_V8 API
myredis | 1:M 02 Oct 2026 11:52:37.184 * Server initialized
myredis | 1:M 02 Oct 2026 11:52:37.184 * Ready to accept connections tcp
myredis | 1:M 02 Oct 2026 11:52:37.184 # WARNING: Redis does not require authentication and is not protected by network restrictions. Redis will accept connections from any IP address on any network interface.
```

Figure 18. Redis starting from the unpinned image: redis. It loads extra modules and RedisTimeSeries reports Redis 8.10.2.



```
syphen@syphen:~/keploy-demo/samples-go/gin-redis$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
743218d648ec   redis:7   "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  0.0.0.0:6379->6379/tcp, [::]:6379->6379
syphen@syphen:~/keploy-demo/samples-go/gin-redis$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
743218d648ec   redis:7   "docker-entrypoint.s..." 9 minutes ago  Up 8 minutes  0.0.0.0:6379->6379/tcp, [::]:6379->6379
syphen@syphen:~/keploy-demo/samples-go/gin-redis$
```

Figure 19. docker ps with the pinned redis:7 container myredis up on port 6379.

```
syphen@syphi: ~/keploy-derr x + v
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Fri Oct 2 17:20:02 IST 2026

System load:  0.0          Processes:      50
Usage of /:   0.3% of 1006.85GB   Users logged in: 2
Memory usage: 10%          IPv4 address for eth0: 172.20.79.55
Swap usage:   0%

* Canonical Workshop gives developers fast, composable, reproducible, and
  secure developer environments that are perfect for agentic workflows.

  https://ubuntu.com/workshop

This message is shown once a day. To disable it please create the
/home/syphen/.hushlogin file.
syphen@syphi:~$ cd ~/keploy-demo/samples-go/gin-redis
syphen@syphi:~/keploy-demo/samples-go/gin-redis$ go build -o gin-redis
ls
Dockerfile  config      controllers  gin-redis  go.sum  img  middlewares  server  structure
README.md   constants  docker-compose.yml  go.mod  helpers  main.go  routes  services  utils
syphen@syphi:~/keploy-demo/samples-go/gin-redis$ |
```

Figure 20. `go build -o gin-redis` produces the `gin-redis` binary (green in the listing).

Newbie tip. Building the binary first matters: Keploy will start it for you in the next step, so you give Keploy the finished program, not `go run`.

Interesting fact. Go compiles your whole program, dependencies included, into a single executable. That is why Keploy can launch it with just `-c ./gin-redis`: no interpreter or runtime to set up.

9. Record: let Keploy watch

9.1 Start the recorder

`-c` is the command that starts your app; Keploy launches it and slips a proxy between your app and Redis.

```
sudo -E env "PATH=$PATH" keploy record -c "./gin-redis"
```

PATH gotcha. The commonly shown form `sudo -E PATH=$PATH keploy ...` failed for me with "sudo: Files/WindowsApps/...: command not found". WSL shares your Windows `PATH`, spaces included, and the unquoted `$PATH` splits at the first space. Quoting it with `env "PATH=$PATH"` fixed it.

Interesting fact. Look at the record log below: Keploy says it started ingress forwarding, so requests to port 3001 are routed through another port (45909 in my run). That is the middleman at work. It also starts its own DNS server, which is why the log warns that it has taken control of DNS resolution.

```

syphen@syphi:~/keploy-demo/samples-go/gin-redis$ sudo -E env "PATH=$PATH" keploy record -c "./gin-redis"
🐰 Keploy: 2026-10-02T17:27:54.25729959+05:30 INFO Starting Keploy {"version": "3.8.58", "apiServerURL": "https://a
pi.keploy.io", "inDocker": false}
API key verified successfully!
✅ You are already on the latest version of Keploy.


FREE EDITION

Keploy Free: 3.8.58

🐰 Keploy: 2026-10-02T17:27:57.401694708+05:30 INFO Generated config file based on the flags that are used
🐰 Keploy: 2026-10-02T17:27:57.401977051+05:30 INFO agent replay posture resolved {"replaying": false, "cmd": "rec
ord", "mode": "", "daemonset": false}
🐰 Keploy: 2026-10-02T17:27:57.613061732+05:30 INFO Starting Keploy {"version": "3.8.58", "apiServerURL": "https://a
pi.keploy.io", "inDocker": false}
🐰 Keploy(agent): 2026-10-02T17:27:57+05:30 INFO agent replay posture resolved {"replaying": false, "cmd": "age
nt", "mode": "record", "daemonset": false}
🐰 Keploy(agent): 2026-10-02T17:27:57+05:30 WARN agent pod cgroup slice unresolved - CPU/RAM/disk metering will r
eport 0 (unavailable) rather than node-wide totals; check that KEPLOY_POD_UID is injected via the downward API
🐰 Keploy(agent): 2026-10-02T17:28:00+05:30 INFO starting UDP DNS server at addr :26789
🐰 Keploy(agent): 2026-10-02T17:28:00+05:30 INFO starting TCP DNS server at addr :26789
🐰 Keploy(agent): 2026-10-02T17:28:00+05:30 INFO Keploy has taken control of the DNS resolution mechanism, your a

```

Figure 21. First keploy record: the banner, then the agent starting up.

```

syphen@syphi: ~/keploy-demo x Windows PowerShell x + v
nnward API
🐰 Keploy(agent): 2026-10-02T19:37:07+05:30 INFO starting UDP DNS server at addr :26789
🐰 Keploy(agent): 2026-10-02T19:37:07+05:30 INFO starting TCP DNS server at addr :26789
🐰 Keploy(agent): 2026-10-02T19:37:07+05:30 INFO Keploy has taken control of the DNS resolution mechani
sm, your application may misbehave if you have provided wrong domain name in your application code.
🐰 Keploy(agent): 2026-10-02T19:37:07+05:30 INFO Proxy started at port:16789
🐰 Keploy(agent): 2026-10-02T19:37:07+05:30 INFO Starting Agent's HTTP server {"addr": "127.0.0.1:61
212"}
🐰 Keploy: 2026-10-02T19:37:07.911361716+05:30 INFO pointing Java at a merged keploy truststore via JAVA_T
OOL_OPTIONS (the JDK's cacerts is left untouched) {"truststore": "/tmp/keploy-java-truststore-0.jks", "s
ystem_source": "/etc/ssl/certs/ca-certificates.crt"}
🐰 Keploy: 2026-10-02T19:37:07.91534472+05:30 INFO Keploy agent is ready to record test cases and mocks.
🐰 Keploy: 2026-10-02T19:37:07.915763003+05:30 INFO Starting Application : {"executing_cmd": "/usr/bin/sh
-c ./gin-redis"}
[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env: export GIN_MODE=release
- using code: gin.SetMode(gin.ReleaseMode)

[GIN-debug] GET /api/getVerificationCode --> github.com/keploy/gin-redis/controllers/GET.VerificationCode
(1 handlers)
[GIN-debug] POST /api/verifyCode --> github.com/keploy/gin-redis/controllers/POST.VerifyCode (1 ha
ndlers)
🐰 Keploy(agent): 2026-10-02T19:37:09+05:30 INFO Started ingress forwarding {"orig_port": 3001, "n
ew_port": 45909}
🐰 Keploy: 2026-10-02T19:37:27.885695595+05:30 INFO ● Keploy has captured test cases for the user's appli
cation. {"path": "/home/syphen/keploy-demo/samples-go/gin-redis/keploy/test-set-5/tests", "testcase name": "ge
t-api-getverificationcode-1"}

```

Figure 22. The app registers both routes, ingress forwarding starts for port 3001, and the first test case (GET) is captured.

9.2 Use the app (Terminal 3)

Make three calls: get an OTP, verify it, then try a wrong one. The first line stores the OTP in a shell variable so you do not have to copy it by hand.

```

OTP=$(curl -s "http://localhost:3001/api/getVerificationCode?email=test@example.com&username=testuser" \
| grep -o '"otp":"[0-9]*"' | grep -o '[0-9]*')

curl -i -X POST http://localhost:3001/api/verifyCode -H "Content-Type: application/json" \
-d '{"email":"test@example.com","otp":"$OTP}" # expect 200 + token

curl -i -X POST http://localhost:3001/api/verifyCode -H "Content-Type: application/json" \
-d '{"email":"test@example.com","otp":1}' # expect 500

curl -i -X POST http://localhost:3001/api/verifyCode -H "Content-Type: application/json" \
-d '{"email":"test@example.com"}' # expect 422

```


Newbie tip. Always check that a `mocks.yaml` exists next to your `tests/` folder before you replay. Tests without mocks are a movie with no sound.

9.4 Detective work: why the recordings broke

Running the recorder with `--debug` showed what happened on Keploy's side:

```
eer close {"Client ConnectionID": "0", "Destination ConnectionID": "1", "Destination Address": "127.0.0.1:6379", "Client IP Address": "127.0.0.1:54256", "parser": "redis", "status": "hung", "shutting_down": false, "error": "supervisor: parser hung beyond 1m0s"}
Keploy(agent): 2026-10-02T20:33:16+05:30 DEBUG Using request-framed HTTP/1.1 proxy with MSG_PEEK pool liveness
Keploy(agent): 2026-10-02T20:33:16+05:30 DEBUG mock marked incomplete {"Client ConnectionID": "0", "Destination ConnectionID": "1", "Destination Address": "127.0.0.1:6379", "Client IP Address": "127.0.0.1:54256", "reason": "paused", "connID": "0"}
Keploy(agent): 2026-10-02T20:33:16+05:30 DEBUG relay: tee drop {"Client ConnectionID": "0", "Destination ConnectionID": "1", "Destination Address": "127.0.0.1:6379", "Client IP Address": "127.0.0.1:54256", "dir": "client", "reason": "paused", "drops_total": 1}
Keploy(agent): 2026-10-02T20:33:16+05:30 DEBUG relay: tee drop {"Client ConnectionID": "0", "Destination ConnectionID": "1", "Destination Address": "127.0.0.1:6379", "Client IP Address": "127.0.0.1:54256", "dir": "dest", "reason": "paused", "drops_total": 1}
Keploy(agent): 2026-10-02T20:33:16+05:30 DEBUG agent: TC suppressed - memory pressure / resync-hole overlapped TC window or a mock was capacity-dropped, not sent to CLI {"tc_name": "", "tc_req_ms": 1790953396380, "tc_resp_ms": 1790953396390, "pressure_overlaps": 0, "resync_orphan_overlaps": 1, "capacity_drop": false, "tcs_suppressed_so_far": 1}
Keploy(agent): 2026-10-02T20:33:29+05:30 DEBUG Using request-framed HTTP/1.1 proxy with MSG_PEEK pool liveness
Keploy(agent): 2026-10-02T20:33:29+05:30 DEBUG relay: tee drop {"Client ConnectionID": "0", "Destination ConnectionID": "1", "Destination Address": "127.0.0.1:6379", "Client IP Address": "127.0.0.1:54256", "dir": "client", "reason": "paused", "drops_total": 2}
Keploy(agent): 2026-10-02T20:33:29+05:30 DEBUG relay: tee drop {"Client ConnectionID": "0", "Destination ConnectionID": "1", "Destination Address": "127.0.0.1:6379", "Client IP Address": "127.0.0.1:54256", "dir": "dest", "reason": "paused", "drops_total": 2}
Keploy(agent): 2026-10-02T20:33:29+05:30 DEBUG agent: TC suppressed - memory pressure / resync-hole overlapped TC window or a mock was capacity-dropped, not sent to CLI {"tc_name": "", "tc_req_ms": 1790953409705, "tc_resp_ms": 1790953409713, "pressure_overlaps": 0, "resync_orphan_overlaps": 1, "capacity_drop": false, "tcs_suppressed_so_far": 2}
Keploy(agent): 2026-10-02T20:33:54+05:30 DEBUG Using request-framed HTTP/1.1 proxy with MSG_PEEK pool liveness
Keploy(agent): 2026-10-02T20:33:54+05:30 DEBUG agent: TC suppressed - memory pressure / resync-hole overlapped TC window or a mock was capacity-dropped, not sent to CLI {"tc_name": "", "tc_req_ms": 1790953434921, "tc_resp_ms": 1790953434923, "pressure_overlaps": 0, "resync_orphan_overlaps": 1, "capacity_drop": false, "tcs_suppressed_so_far": 3}
Keploy(agent): 2026-10-02T20:34:04+05:30 DEBUG Using request-framed HTTP/1.1 proxy with MSG_PEEK pool liveness
Keploy: 2026-10-02T20:34:04.504054889+05:30 DEBUG Starting test case encoding {"kind": "Http", "name": ""}
Keploy: 2026-10-02T20:34:04.504178967+05:30 DEBUG Encoding HTTP test case
Keploy: 2026-10-02T20:34:04.524593108+05:30 INFO ● Keploy has captured test cases for the user's application. {"path": "/home/syphen/keploy-demo/samples-go/gin-redis/keploy/test-set-0/tests", "testcase name": "post-api-verifycode-1"}
Keploy(agent): 2026-10-02T20:34:19+05:30 DEBUG Using request-framed HTTP/1.1 proxy with MSG_PEEK pool liveness
Keploy: 2026-10-02T20:34:19.612868194+05:30 DEBUG Starting test case encoding {"kind": "Http", "name": ""}
Keploy: 2026-10-02T20:34:19.61307699+05:30 DEBUG Encoding HTTP test case
Keploy: 2026-10-02T20:34:19.61501937+05:30 INFO ● Keploy has captured test cases for the user's application. {"path": "/home/syphen/keploy-demo/samples-go/gin-redis/keploy/test-set-0/tests", "testcase name": "post-api-verifycode-2"}
```

Figure 25. Debug log: the supervisor stops the Redis parser ("parser hung beyond 1m0s") and falls back to passthrough. The calls that touched Redis are logged as "TC suppressed"; only the two 422 calls, which never reach Redis, were saved (test-set-0).

Session	Redis image	Gap before a Redis call	Result
17:27 first attempt	Unpinned (8.x)	about 72 s after app start	"parser retired ... hung"; tests left out
19:37, test-set-5	not recorded	2 min 5 s between the GET and the first POST	4 tests, no mocks.yaml
20:30, test-set-0	redis:7 (7.4.11)	about 3 min after app start	Handshake mocks only; two 422 tests saved
20:38, test-set-1	redis:7 (7.4.11)	GET 13 s after start, POST 32 s later	3 tests and 6 Redis mocks (worked)

A correction. My first suspect was the unpinned Redis 8.x image, so I pinned `redis:7`. The data says that was not the fix: the 20:30 session already ran Redis 7.4.11 and still stalled. What does line up across all four sessions is **idle time**: the Redis connection sat quiet for over a minute before the next call in every failed session, but not in the one that worked, and the log itself says the parser "hung beyond 1m0s". This is a hypothesis, not a confirmed cause. Easy test: record again, wait two minutes between the GET and the POST, and see whether the mocks go missing.

Interesting fact. Redis clients such as `go-redis` keep a pool of connections open and reuse them, so a connection can sit idle between your requests. That is why the gap between my calls was worth checking.

9.5 The recording that worked

Same Redis container, same command, but every call sent within about half a minute of starting the app.

```
0.0.1:46707"}
Keploy(agent): 2026-10-02T20:38:46+05:30 DEBUG Started Ingress forwarder {"listening_on": "0.0.0.0:3001", "forwarding_to": "127.0.0.1:46707"}
Keploy(agent): 2026-10-02T20:38:46+05:30 INFO Started ingress forwarding {"orig_port": 3001, "new_port": 46707}
Keploy(agent): 2026-10-02T20:38:59+05:30 DEBUG Using request-framed HTTP/1.1 proxy with MSG_PEEK pool liveness
Keploy: 2026-10-02T20:38:59.591659769+05:30 DEBUG Starting test case encoding {"kind": "Http", "name": ""}
Keploy: 2026-10-02T20:38:59.592065763+05:30 DEBUG Encoding HTTP test case
Keploy: 2026-10-02T20:38:59.593725264+05:30 INFO ● Keploy has captured test cases for the user's application. {"path": "/home/syphen/keploy-demo/samples-go/gin-redis/keploy/test-set-1/tests", "testcase name": "get-api-getverificationcode-1"}
```

Figure 26. Working session: ingress started at 20:38:46 and the GET was captured 13 seconds later into test-set-1.

Why I like this. Even when recording went wrong, Keploy did not hide it. The warnings and the debug log pointed at the exact moment and component, which is what let me find the pattern. A tool that explains its own failures is worth a lot.

Result: three tests (GET 200, POST 200 with a token, POST 500 for the wrong OTP) and a mocks file with 6 Redis mocks, including the SET. When recording stopped, Keploy ran an automatic replay that reported 1 passed and 2 failed, which the random fields in section 11 explain.

10. What Keploy wrote: just text files

```
keploy/  
test-set-0/ 2 tests (the two 422 cases) + mocks.yaml (Redis handshake only)  
test-set-1/ 3 tests + mocks.yaml (6 Redis mocks) + mappings.yaml  
  tests/get-api-getverificationcode-1.yaml -> 200, returns the OTP  
  tests/post-api-verifycode-1.yaml         -> 200, returns a JWT  
  tests/post-api-verifycode-2.yaml         -> 500, wrong OTP  
reports/
```

Open a test and there is no magic. It is the request, the answer it expects, and a list of fields to ignore (shortened):

```
req:  
  method: GET  
  url: http://localhost:3001/api/getVerificationCode?email=test@example.com&username=testuser  
resp:  
  status_code: 200  
  body: '{"status":"true","message":"OTP Generated successfully","otp":"4123"}'  
assertions:  
  noise:  
    body.otp: []  
    header.Date: []
```

The mocks file holds the Redis conversation as raw protocol messages: the go-redis hello (hello, client set info) and the write that stores the OTP:

```
SET test@example.com {"otp":"4123","username":"testuser"} EX 14400
```

Interesting fact. The odd-looking text inside mocks.yaml is Redis's wire protocol, RESP, and it is human-readable. *5 means an array of five items, and \$16 means the next string is 16 bytes long (test@example.com is 16 characters). The first handshake mock is HELLO 3: go-redis asking Redis to speak the newer RESP3.

Newbie tip. Keploy runs under sudo, so it creates these files as root. To edit them: `sudo chown -R $USER:$USER keploy`.

11. Replay: the stunt double steps in

Two values change on every run, so an exact comparison would fail even though the app is healthy. This is where "noise" comes in: you tell Keploy to ignore those fields.

Field	Why it changes	Fix in that test's YAML
body.otp (GET)	A random 4-digit number on every call	assertions.noise: body.otp: []
body.token (success POST)	A JWT whose payload has an expiry exactly 14,400 seconds after creation	assertions.noise: body.token: []

Interesting fact. A JWT has three dot-separated parts: header, payload and signature. The payload is only encoded, not encrypted, so anyone can read it. Decoding this app's token shows an exp claim, an expiry time that is different for every token, which is exactly why body.token has to be noise. The signature only proves nobody tampered with it.

header.Date was already marked as noise by Keploy for me. Now the big moment: switch Redis off, then replay.

```
docker compose down
docker ps
sudo -E env "PATH=$PATH" keploy test -c "./gin-redis" --delay 10
```

```
d
myredis | 1:C 02 Oct 2026 15:00:02.887 # Warning: no config file specified, using the default config. In order to speci
fy a config file use redis-server /path/to/redis.conf
myredis | 1:M 02 Oct 2026 15:00:02.888 * monotonic clock: POSIX clock_gettime
myredis | 1:M 02 Oct 2026 15:00:02.889 * Running mode=standalone, port=6379.
myredis | 1:M 02 Oct 2026 15:00:02.893 * Server initialized
myredis | 1:M 02 Oct 2026 15:00:02.893 * Ready to accept connections tcp
myredis | 1:M 02 Oct 2026 16:00:03.002 * 1 changes in 3600 seconds. Saving...
myredis | 1:M 02 Oct 2026 16:00:03.006 * Background saving started by pid 16
myredis | 16:C 02 Oct 2026 16:00:03.016 * DB saved on disk
myredis | 16:C 02 Oct 2026 16:00:03.017 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
myredis | 1:M 02 Oct 2026 16:00:03.107 * Background saving terminated with success
Gracefully Stopping... press Ctrl+C again to force
Container myredis Stopping
myredis | 1:signal-handler (1790972558) Received SIGTERM scheduling shutdown...
myredis | 1:M 02 Oct 2026 20:22:38.781 * User requested shutdown...
myredis | 1:M 02 Oct 2026 20:22:38.781 * Saving the final RDB snapshot before exiting.
myredis | 1:M 02 Oct 2026 20:22:38.789 * DB saved on disk
myredis | 1:M 02 Oct 2026 20:22:38.790 # Redis is now ready to exit, bye bye...
Container myredis Stopped
myredis exited with code 0
syphen@syphi:~/keploy-demo/samples-go/gin-redis$ docker compose down
docker ps
WARN[0000] /home/syphen/keploy-demo/samples-go/gin-redis/docker-compose.yml: the attribute 'version' is obsolete, it wil
l be ignored, please remove it to avoid potential confusion
[+] down 2/2
✔Container myredis Removed 0.1s
✔Network gin-redis_default Removed 0.3s
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
syphen@syphi:~/keploy-demo/samples-go/gin-redis$
```

Figure 27. Redis stopped and removed (docker compose down), and docker ps shows no containers.

And the result:

```
COMPLETE TESTRUN SUMMARY.
Total tests: 5      Total test passed: 5      Total test failed: 0      Total time taken: "20.10 s"
"test-set-0"      2      2      0      "10.05 s"
"test-set-1"      3      3      0      "10.05 s"
```

Interesting fact. Each test set reports 10.05 s, and 10 of those seconds are the --delay 10 we set so the app can start. The replay itself added roughly 0.05 s per set.

Why I like this. Redis was switched off, deleted even, and the app still passed every test. Keploy answered the app's Redis calls from the recorded mocks, so the tests do not need Redis, a Docker network, or any seed data. That is the whole pitch in one screenshot: **no test code, no hand-written mocks, no dependencies to babysit.**

12. Your turn: break it on purpose

A test that never fails is just a decoration. So let us make one fail on purpose and see Keploy catch it. (This is the one step I have not run yet, so the screenshots are pending.)

```
# 1. Change the success message
nano controllers/POST/postVerificationCode.go      # "OTP authenticated successfully" -> "OTP verified"

# 2. Rebuild and replay
go build -o gin-redis
sudo -E env "PATH=$PATH" keploy test -c "./gin-redis" --delay 10      # expect: the success-POST test fails

# 3. Undo it
git checkout controllers/POST/postVerificationCode.go
go build -o gin-redis
sudo -E env "PATH=$PATH" keploy test -c "./gin-redis" --delay 10      # expect: all green again
```

Why this matters. This is the everyday use. Change code, run `keploy test`, and a changed response shows up straight away, with a diff, instead of in production.

Interesting fact. A failing replay is easy to trust. Keploy feeds the app the same recorded Redis answers every time, so when a test fails, the change is in your code or your response, not in the infrastructure.

13. Cheat sheet: everything in order

All the commands from this guide in one place. Terminals are numbered like section 8.

```
# One-time setup (Ubuntu terminal)
curl --silent -O -L https://keploy.io/install.sh && source install.sh
keploy login --api-key <YOUR_API_KEY>
mkdir -p ~/keploy-demo && cd ~/keploy-demo
git clone https://github.com/keploy/samples-go.git
cd samples-go/gin-redis && go mod download && go build -o gin-redis

# Terminal 1
docker compose up redis

# Terminal 2: record
sudo -E env "PATH=$PATH" keploy record -c "./gin-redis"

# Terminal 3: use the app (GET, then POST with the OTP, then a wrong OTP; see section 9.2)
# Terminal 2: Ctrl+C to stop recording, then check that mocks.yaml exists
find keploy -type f
sudo chown -R $USER:$USER keploy

# Add noise (body.otp, body.token), then switch Redis off and replay
docker compose down
sudo -E env "PATH=$PATH" keploy test -c "./gin-redis" --delay 10
```

14. Troubleshooting log

Every bump I hit, so you do not have to.

Symptom	Cause	Fix
keploy.exe vanishes after download; Trojan warning (native Windows)	Windows Security flagged the file; not verified as a false positive	I did not allow it; used WSL2 instead
Failed to start the logger ... chmod keploy-logs.txt: operation not permitted	Running from a Windows-mounted folder (/mnt/d/...)	Work in the Linux home folder (~/.keploy-demo)
go: command not found	Snap Go; /snap/bin not on PATH	Add /snap/bin to PATH in ~/.bashrc
docker not found in WSL	Docker Desktop's WSL integration is off	Enable it in Docker Desktop, reopen the terminal
permission denied on docker.sock	User not in the docker group	sudo usermod -aG docker \$USER; newgrp docker
sudo: Files/WindowsApps/...: command not found	Windows PATH has spaces; unquoted \$PATH splits	sudo -E env "PATH=\$PATH" keploy ...
Tests saved without mocks (or mocks without tests); log says parser hung beyond 1m0s	Redis parser stalled. Leading hypothesis: connection idle for over a minute (unconfirmed). Pinning redis:7 alone did not fix it	Re-record with all calls close together; confirm mocks.yaml exists before replaying
Replay: no mock matched ...; app exits with Redis EOF	That test set had no Redis mocks	Replay a set that contains mocks.yaml
Replay fails on random OTP or JWT	Values change every run	Add body.otp and body.token to assertions.noise
Cannot edit files in keploy/	Created by sudo, owned by root	sudo chown -R \$USER:\$USER keploy

15. Why Keploy earned a thumbs up

After a day of installing, breaking and fixing things, here is what stuck with me:

- **It made tests out of normal use.** Three curl calls turned into three tests. I wrote no test code.
- **It handled the dependency for me.** The Redis conversation, down to the exact SET with its 4-hour expiry, was saved and replayed. Redis could be off.
- **The output is honest and readable.** Plain YAML tests, clear log lines, and debug output that explained the one thing that went wrong.
- **It fits how Go developers already work.** Build your binary, hand it to Keploy. No SDK, no app code changes in my run.
- **The free plan covered all of it.** Nothing in this guide needed a paid tier.

Why I like this. If you have ever skipped writing API tests because faking Redis or a database felt like a second job, this is the tool that makes that excuse go away. It turned a chore into a few commands.

Next things I want to try

Scrolling the `keploy` command list from the install screenshot, three commands caught my eye: `export` and `import` (Postman collections) and `diff` (compare two test runs). I have not tried them yet. More in the Keploy docs at keploy.io/docs, and more Go samples in github.com/keploy/samples-go.

Appendix A. Figure index

Maps each figure to the original screenshot (by capture time) and the suggested file name for the website repo.

Fig	Original screenshot	Suggested name	Section
1	Screenshot_2026-10-02_153650.png	01a-quickstart-language.png	4
2	Screenshot_2026-10-02_153700.png	01b-quickstart-environment.png	4
3	Screenshot_2026-10-02_153722.png	01-quickstart-list.png	4
4	Screenshot_2026-10-02_153753.png	02a-native-install-docs.png	5.1
5	Screenshot_2026-10-02_155642.png	troubleshoot-native-install-vanished.png	5.1
6	Screenshot_2026-10-02_155824.png	02-install-methods.png	5.2
7	Screenshot_2026-10-02_155732.png	wsl-install-progress.png	5.2
8	Screenshot_2026-10-02_155926.png	wsl-installed.png	5.2
9	Screenshot_2026-10-02_155832.png	02b-wsl-install-docs.png	5.2
10	Screenshot_2026-10-02_160104.png	03-keploy-version.png	5.2
11	Screenshot_2026-10-02_160318.png	04a-login-docs.png	5.3
12	Screenshot_2026-10-02_160535.png	04b-api-keys-page.png	5.3
13	Screenshot_2026-10-02_161602.png	04-keploy-login.png	5.3
14	Screenshot_2026-10-02_161825.png	troubleshoot-go-docker-missing.png	6
15	Screenshot_2026-10-02_170315.png	tools-go-docker-ok.png	6
16	Screenshot_2026-10-02_170747.png	05-clone.png	7
17	Screenshot_2026-10-02_170941.png	06c-compose-file.png	7
18	Screenshot_2026-10-02_172257.png	troubleshoot-redis8-log.png	8
19	Screenshot_2026-10-02_194240.png	07-redis-running.png	8
20	Screenshot_2026-10-02_172313.png	08-build.png	8
21	Screenshot_2026-10-02_172816.png	09-record-start.png	9.1
22	Screenshot_2026-10-02_194153.png	09b-record-app-started.png	9.1
23	Screenshot_2026-10-02_203516.png	10-api-calls.png	9.2
24	Screenshot_2026-10-02_194327.png	11-record-captured.png	9.3
25	Screenshot_2026-10-02_203534.png	troubleshoot-redis-parser-hung.png	9.4
26	Screenshot_2026-10-02_203914.png	11b-record-captured-working.png	9.5
27	Screenshot_2026-10-03_015311.png	15-redis-stopped.png	11

Appendix B. Screenshots still needed

Name	What to capture	Section
tools-versions.png	One terminal view with go, docker, compose, git, uname -r and keploy versions all working.	3
troubleshoot-defender-trojan.png	Windows Security, Protection history: the entry with the exact detection name.	5.1
06-routes-in-code.png	routes/routes.go, showing the two routes.	7
06b-redis-address.png	The Addr: "localhost:6379" line in helpers/redis/redisConnect.go.	7
12-keploy-folder-tree.png	Output of: find keploy -type f	10
13-test-yaml.png	The GET test YAML in your editor or via cat.	10
14-mocks-yaml.png	The Redis SET mock in mocks.yaml.	10
16a-test-fail-noise.png	The first replay failing on body.otp / body.token (from the auto-replay report).	11
16b-noise-edit.png	The assertions.noise block after adding the fields.	11
16-test-pass.png	The 5 of 5 summary above, as a screenshot (crop the taskbar).	11
17-code-change.png	The edit (git diff or your editor).	12
18-test-fail.png	The failing report showing the diff on the success POST.	12
19-test-pass-again.png	All tests passing again after the revert.	12
troubleshoot-wsl-chmod.png	The chmod keploy-logs.txt error.	14
troubleshoot-docker-permission.png	The docker.sock permission denied error.	14
troubleshoot-sudo-path.png	The sudo PATH command-not-found error.	14

Appendix C. From guide to website

How this document maps to the assignment, and what is left.

Assignment step	Status
Run a Keploy Go quickstart	Done: Gin + Redis, record and replay, 5 of 5 tests passing
Document it for a developer new to Keploy Show a failing test (the a-ha moment) — Not yet run	This guide. For the web page, tighten it to a crisp, step-by-step voice
Next.js site with MDX	Next: App Router, @next/mdx, custom components (Callout, Steps, code block with copy button, OS tabs, flow diagram, troubleshooting accordion), dark/light toggle
Public GitHub repository	Next: new repo with only the website code and images (no samples-go, no keploy/recordings, no API keys)
Deploy on Vercel	Next: import the repo, default Next.js settings
Deliverables	Two links: the GitHub repo and the live Vercel URL

How the sections become the page: sections 1 and 2 are the intro, 3 to 12 become the numbered steps, 13 is a copyable cheat sheet, 14 is a troubleshooting accordion, and 15 is the takeaway and next steps.